

# **LIBRAX**

## Smart Library Management System

### **PROJECT REPORT**

#### **CORE JAVA PROJECT**

A standalone library management application with command-line and Java Swing interfaces, file-based persistence, circulation management, and analytics.

SUBMITTED BY: - MD FARHAN ALI

REGISTRATION NUMBER: - 25BAI10792

SUBMITTED TO: - VIT Bhopal

ACADEMIC YEAR: - 2026-2027

# INTRODUCTION

Libraries manage several connected activities at the same time: maintaining book records, registering members, issuing and returning books, tracking due dates, and producing operational reports. When these activities are handled manually, searching records and maintaining consistent circulation information can become time-consuming and prone to data-entry errors.

LibraX is a standalone Core Java application developed to model these operations in a structured software system. It provides two interaction modes: a command-line application for direct operation and a lightweight Java Swing dashboard for visual access to common functions. The application does not depend on JDBC, MySQL, or an external database.

The system separates the domain model, business logic, persistence, reporting, exceptions, and user interfaces into packages. Book, member, librarian, and transaction information is represented through Java classes and collections. Data is persisted through Java object serialization into local .dat files, allowing records to remain available between executions.

The project combines object-oriented modeling, collections, exception handling, file I/O, serialization, and multithreading in one realistic workflow. The current demonstration dataset contains 40 books, 20 members, 3 librarians, and 25 transactions. The dashboard reports 32 available books, 8 issued books, 3 overdue loans, INR 75.00 collected fines, and 80% availability.

# PROBLEM STATEMENT

A small or academic library needs a consistent way to maintain its catalogue, member records, circulation history, due dates, and reports. A manual register-based workflow makes it difficult to search records quickly, prevent invalid circulation operations, enforce borrowing limits, and obtain a consolidated view of the library's current state.

The problem addressed by LibraX is to design and implement a modular library management system that manages books and members, performs issue/return operations, calculates overdue fines, preserves records between executions, and presents useful operational analytics through Core Java.

## OBJECTIVES

- Develop a complete library management workflow using Core Java concepts.
- Provide book management operations including addition, update, removal, listing, and search.
- Register members with different membership types and borrowing limits.
- Support issue and return operations with due-date and fine calculation.
- Persist library records locally and reload them at startup.
- Generate analytics for availability, overdue loans, categories, borrowing activity, and fines.
- Provide both command-line execution and a lightweight Swing dashboard.

# FUNCTIONAL REQUIREMENTS

The project contains multiple major functional modules and maintains a clear input → processing → output workflow.

| Module            | Functions                                                | Input → Output                    |
|-------------------|----------------------------------------------------------|-----------------------------------|
| Book Management   | Add, update, remove, list, search; availability tracking | Book details → book/status        |
| Member Management | Register and view members; membership tiers              | Member details → profile/limit    |
| Circulation       | Issue, return, active loans, overdue loans               | Book ID + Member ID → transaction |
| Search            | Search by ID, title, author, category                    | Query → matching books            |
| Reports           | Inventory, overdue, fines, categories, activity          | Records → analytics/report        |
| Persistence       | Load/save all major collections                          | Collections ↔ .dat files          |
| Swing Dashboard   | Dashboard, books, members, circulation, reports          | GUI actions → visual output       |

## INPUT / OUTPUT STRUCTURE

- Book input: ID, title, author, category → stored book and status.
- Member input: ID, personal details, membership type → member profile and borrowing capacity.
- Circulation input: book ID and member ID → transaction, due date, status, and fine.
- Report output: summary metrics, overdue information, category statistics, activity, and exported report text.

## **NON-FUNCTIONAL REQUIREMENTS**

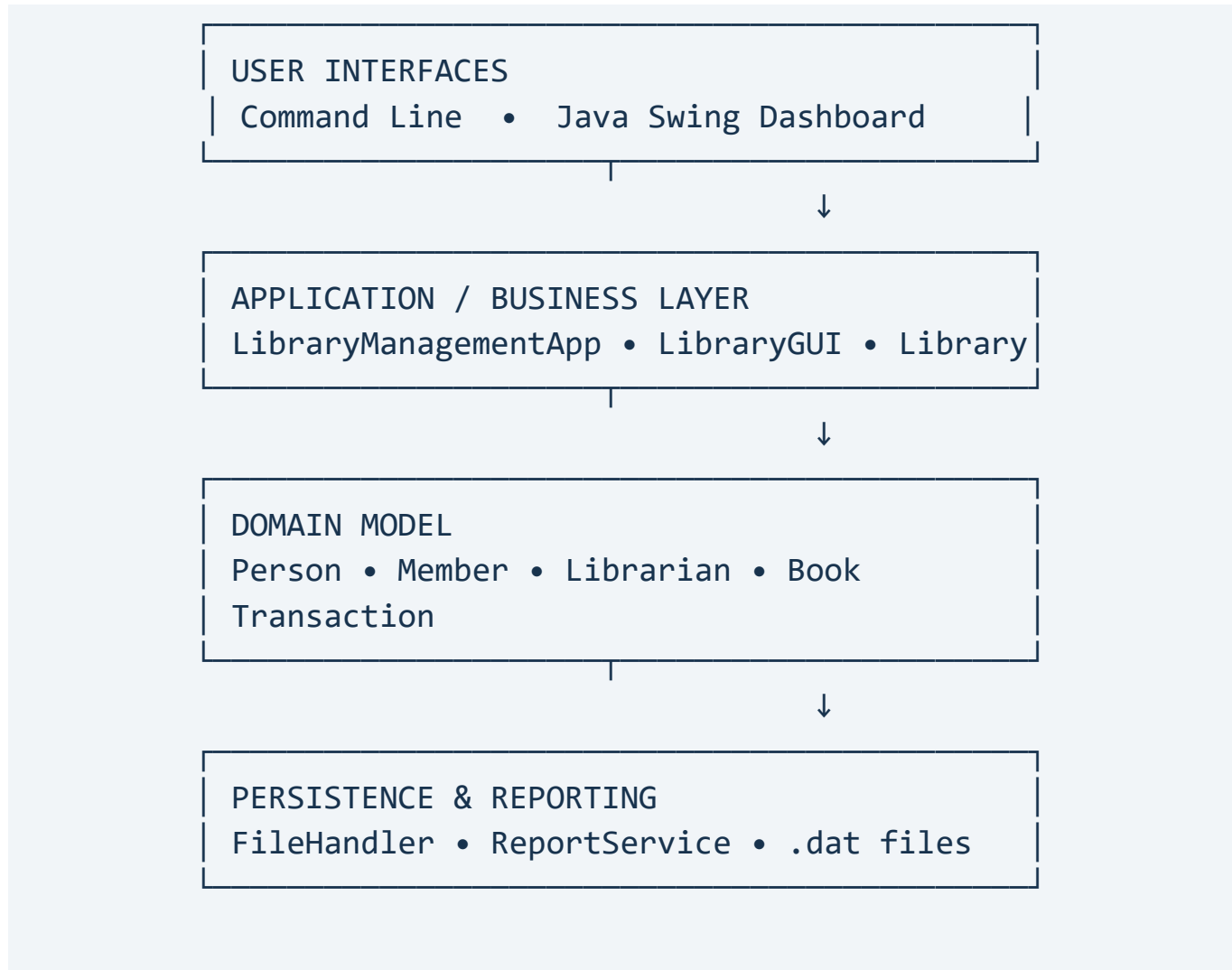
- Performance: common in-memory operations should remain responsive for the intended dataset size.
- Usability: numbered CLI menus and a grouped Swing dashboard provide clear access to operations.
- Reliability: circulation updates Book, Member, and Transaction state together.
- Maintainability: app, model, service, and exception packages separate responsibilities.
- Error Handling: custom exceptions address invalid input, missing members, and unavailable books.
- Resource Efficiency: collections provide simple dynamic storage and reporting can run in a separate thread.
- Portability: the application requires a standard JDK and local files, without a database server.
- Persistence: serialized .dat files preserve records across application restarts.

## **TECHNICAL CONSTRAINTS**

- No JDBC/MySQL or external database is used.
- Local serialized files are intended for a small-library/academic scenario rather than multi-user deployment.
- The current implementation assumes a single local application instance and does not provide network authentication.

## SYSTEM ARCHITECTURE

LibraX uses a simple layered architecture. The user-facing layer contains the command-line application and Swing dashboard. These interfaces call the Library business layer, which coordinates domain objects. FileHandler and ReportService provide persistence and reporting support.

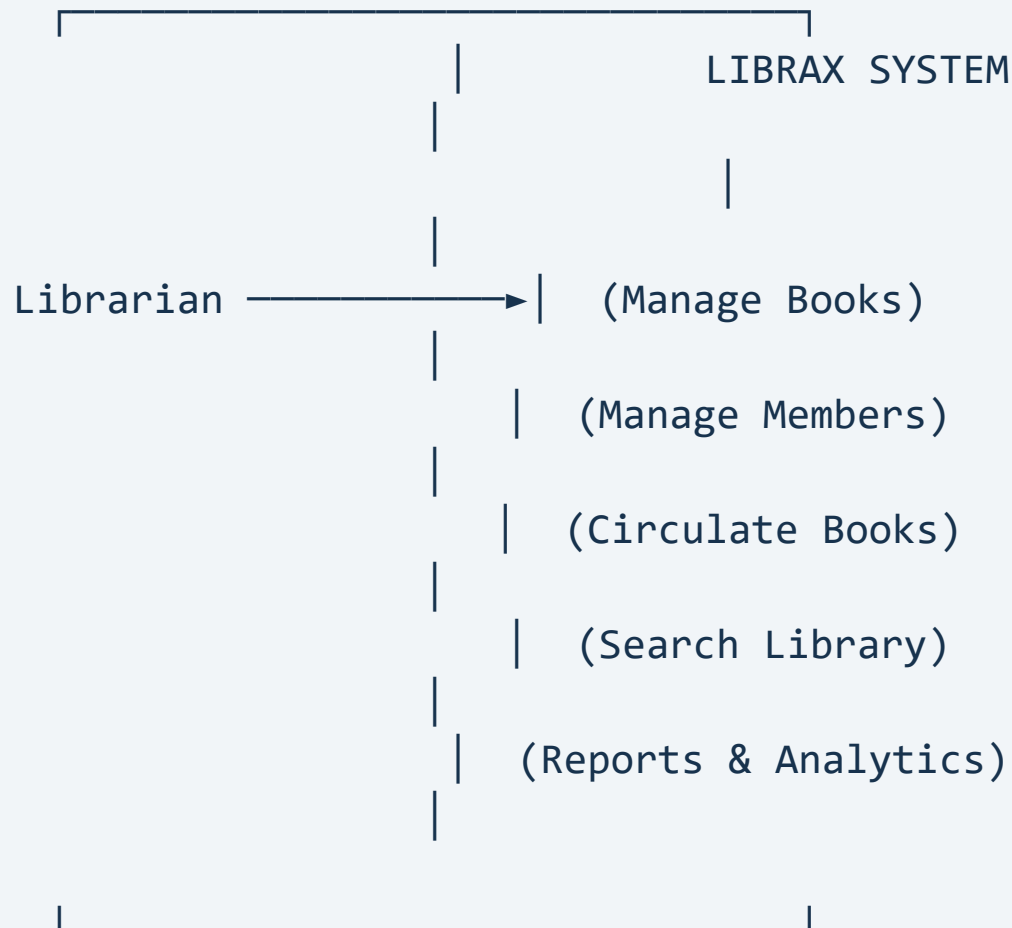


*Figure 1. Layered architecture of LibraX.*

The core Library service is shared by both presentation modes, avoiding duplicated circulation logic. The package structure is: app for entry points/UI, model for domain entities, service for business/persistence/reporting, and exception for custom exceptions.

## USE CASE DIAGRAM

The primary actor is the Librarian. The librarian maintains the catalogue, registers members, performs circulation, searches the library, and generates reports. These responsibilities are accessible through the CLI and, for common operations, the Swing interface.



*Figure 2. Use case view of the Librax system.*

# WORKFLOW DIAGRAM

## SEQUENCE DIAGRAM

The following sequence represents the issue-book operation. The librarian supplies the book and member identifiers. The Library service validates the entities, updates the Book and Member objects, creates a Transaction, and requests persistence.

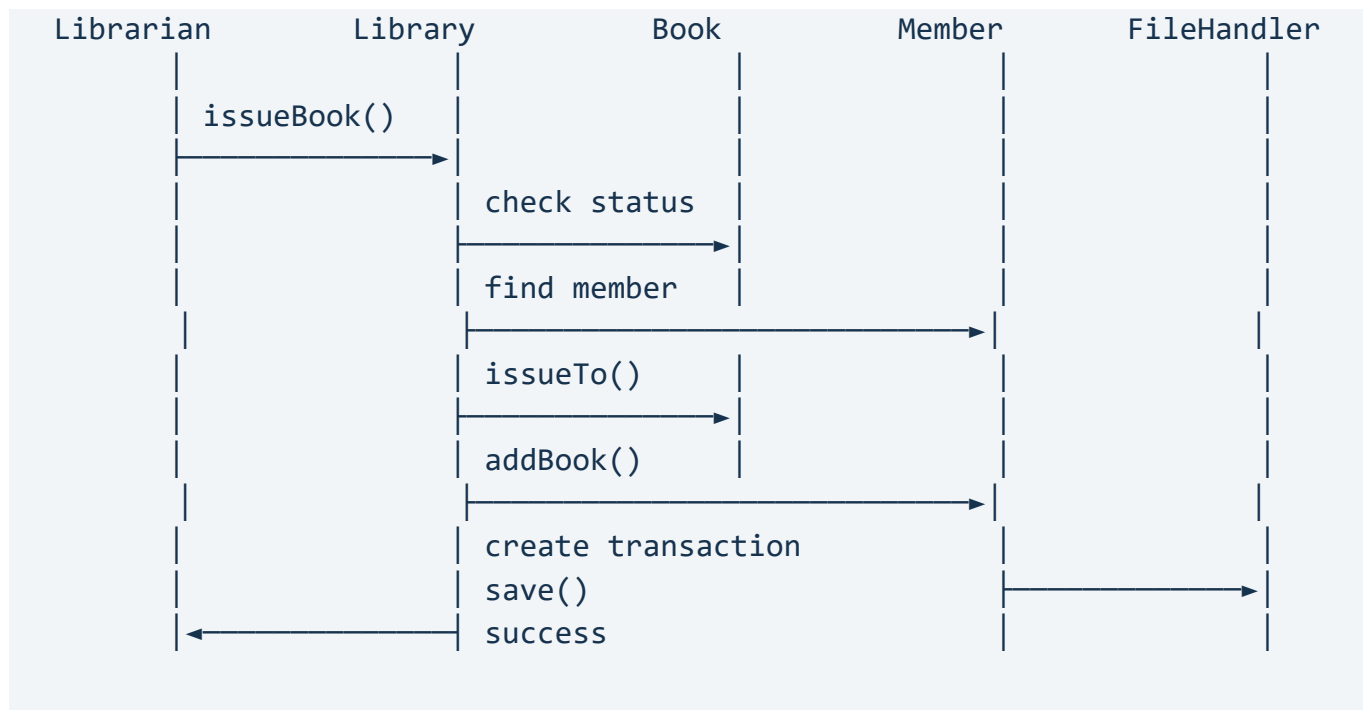


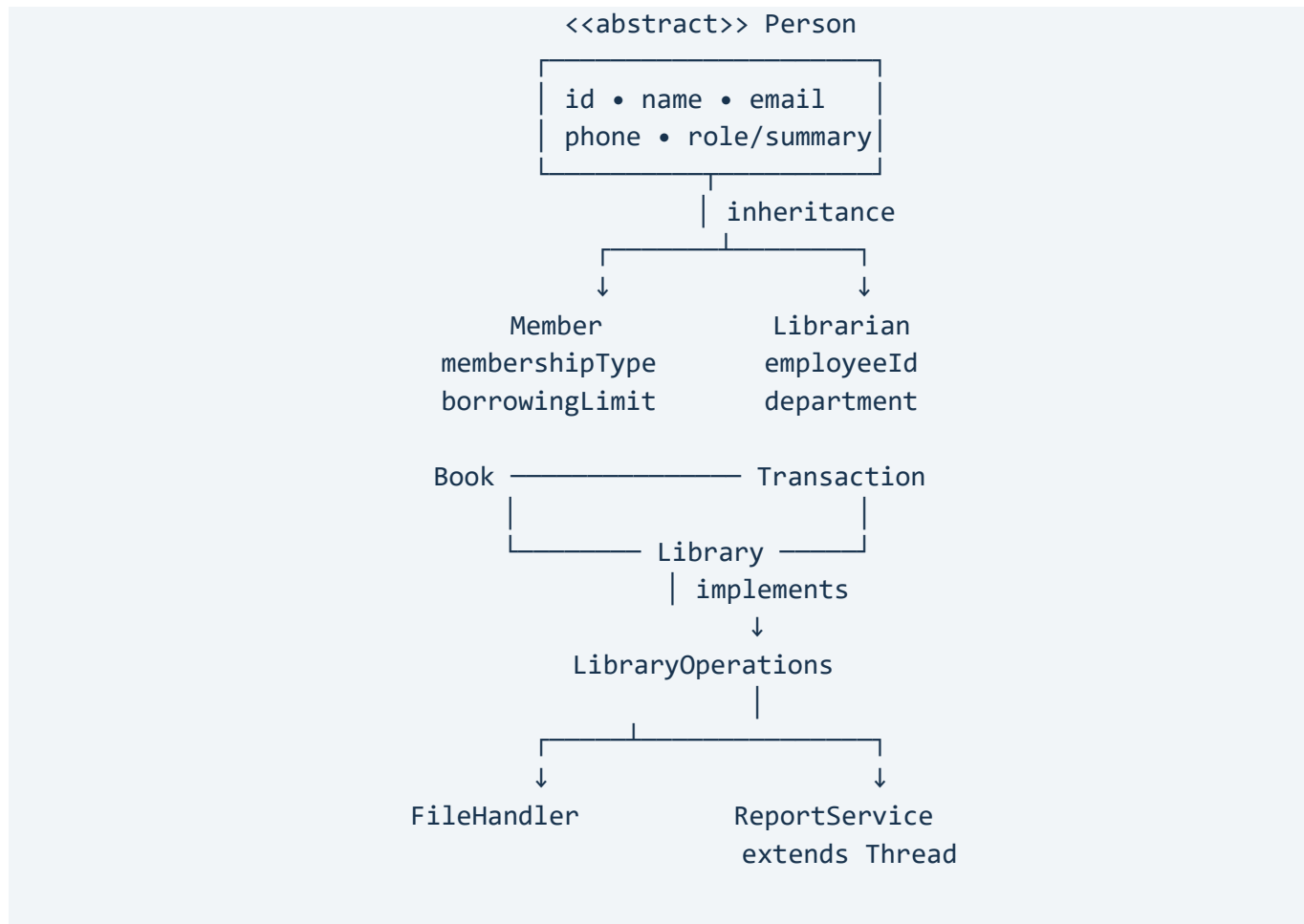
Figure 4. Sequence for issuing a book.

The sequence demonstrates the separation between interface code and the central business layer. FileHandler remains responsible for persistence so file-specific logic is not repeated throughout the application.



## CLASS DIAGRAM

The object model uses abstraction, inheritance, encapsulation, interfaces, and associations. Person is an abstract base class; Member and Librarian extend it. Library implements LibraryOperations and manages collections of the principal domain entities.



*Figure 5. Simplified class structure of LibraX.*

`Book.Status` represents AVAILABLE/ISSUED states. `Member.MembershipType` represents STUDENT, REGULAR, and PREMIUM levels with different borrowing limits. `Transaction` uses Java time classes to calculate overdue days and fines.

## DESIGN DECISIONS & RATIONALE

Modular design: app, model, service, and exception packages keep responsibilities organized and improve maintainability.

Object-oriented domain model: books, people, and transactions are represented as objects so the real-world workflow maps naturally to Java classes.

Abstract Person hierarchy: common identity/contact fields are shared while Member and Librarian retain role-specific behavior.

Interface contract: LibraryOperations defines a clear contract for core library actions and Library supplies the implementation.

List-based collections: ArrayList/List collections provide dynamic storage for the application's books, members, librarians, and transactions.

File-based serialization: local .dat files provide persistence without JDBC or an external database, keeping the project self-contained.

Custom exceptions: domain-specific error types make common invalid states explicit and easier to handle.

Multithreading: ReportService extends Thread so analytics generation can be initiated as a background task.

Dual interface: the CLI remains the complete executable workflow while Swing provides a visual demonstration using the same business layer.

# IMPLEMENTATION DETAILS

## SCREENSHOTS AND RESULTS

### Command-Line Interface

```
Enter choice: 4
Search by ID, title, author or category: Java

ID      | TITLE                      | AUTHOR          | CATEGORY    | STATUS
-----|-----|-----|-----|-----
B101    | Java: The Complete Reference | Herbert Schildt  | Programming | AVAILABLE
B104    | Effective Java               | Joshua Bloch    | Programming | AVAILABLE
B105    | Head First Java              | Kathy Sierra    | Programming | AVAILABLE
Records shown: 3

----- MAIN MENU -----
1. Book Management
2. Member Management
3. Issue / Return
4. Search Library
5. Reports & Analytics
6. Open Swing Dashboard
7. Save Data
0. Exit
-----
```

Figure 6. CLI search and main menu. The search for “Java” returns three programming books, after which the main menu is displayed.

The CLI demonstrates direct execution of the library workflow. It provides numbered options for book management, member management, issue/return, search, reports, the Swing dashboard, saving, and exit. The screenshot also demonstrates attribute-based book search.

# SCREENSHOTS AND RESULTS

## Java Swing Dashboard

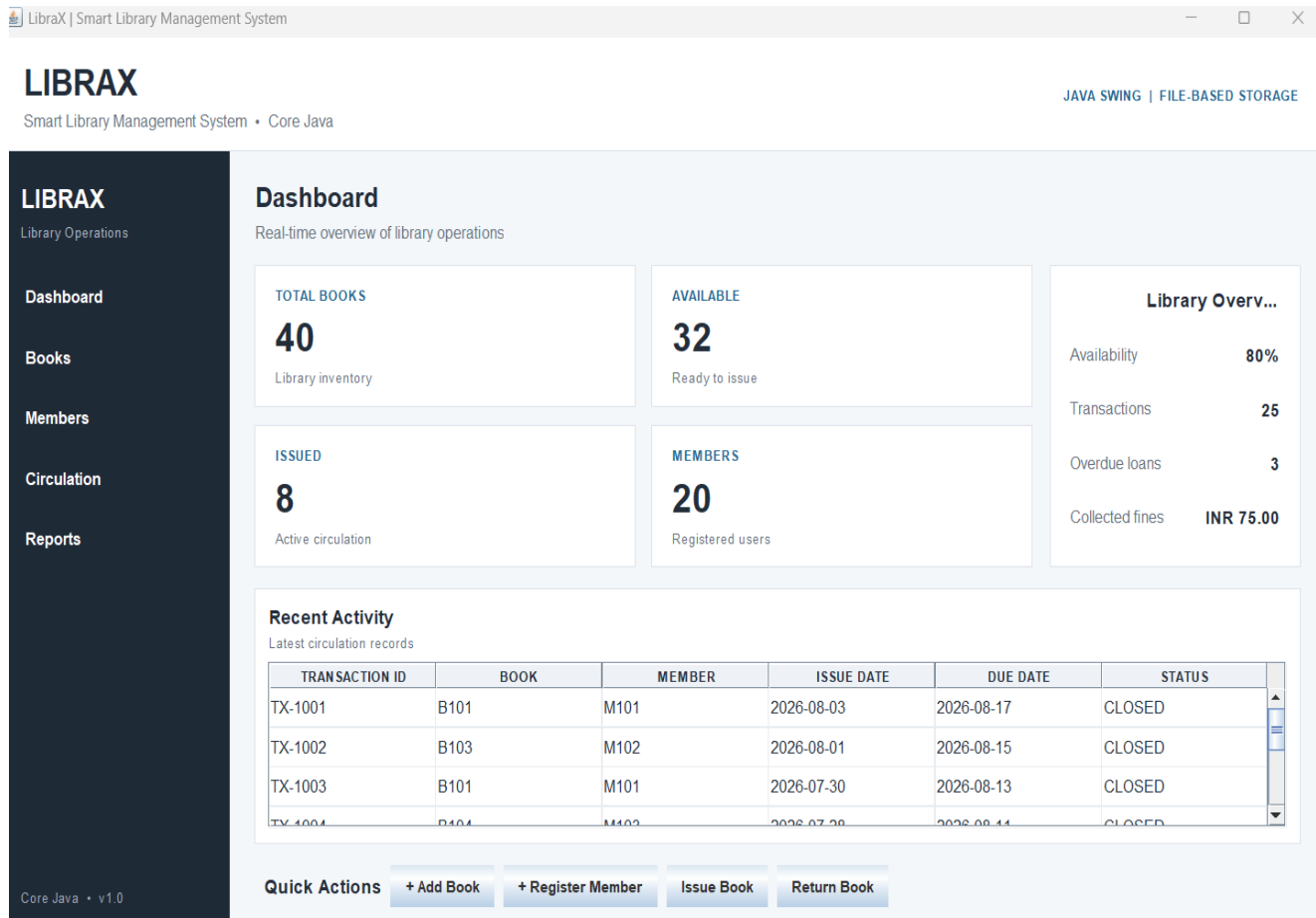


Figure 7. Swing dashboard showing the prepared LibraX dataset.

The dashboard reports 40 total books, 32 available, 8 issued, and 20 members. The overview reports 80% availability, 25 transactions, 3 overdue loans, and INR 75.00 collected fines. Recent activity is shown as a transaction table with book/member identifiers, dates, and status.

These values demonstrate that the GUI is connected to the application data and provides a compact visual summary of the library's current operational state.

## TEST APPROACH

Testing focused on the major workflows, validation paths, persistence, reporting, and the Swing interface. The project uses practical smoke/validation testing rather than an external JUnit suite.

| ID   | Scenario                        | Expected Result                                              |
|------|---------------------------------|--------------------------------------------------------------|
| TC01 | Start with prepared data        | Books, members and transactions load.                        |
| TC02 | Search by title/author/category | Matching records are displayed.                              |
| TC03 | Add unique book                 | Book is added and can be saved.                              |
| TC04 | Duplicate book ID               | Invalid duplicate is rejected.                               |
| TC05 | Issue available book            | Book/member/transaction state is updated; due date created.  |
| TC06 | Issue unavailable book          | Operation is rejected with exception/validation.             |
| TC07 | Return borrowed book            | Book becomes available; transaction closes; fine calculated. |
| TC08 | Invalid return                  | Invalid circulation operation is rejected.                   |
| TC09 | Generate report                 | Analytics are calculated and exported.                       |
| TC10 | Restart application             | Saved data is loaded again.                                  |
| TC11 | Open Swing dashboard            | Current statistics are displayed.                            |

## VALIDATION OBSERVATIONS

- The application was compiled and launched successfully during development.
- The prepared dataset loaded into the application and appeared in the Swing dashboard.
- The CLI search shown in Figure 6 returned three matching Java-related books.
- The reporting/analytics workflow was exercised during development.

The matrix describes the intended validation coverage; it is not presented as a formal JUnit test report.

## CHALLENGES FACED

- Circulation consistency: issuing and returning a book changes Book, Member, and Transaction state together.
- Persistence: multiple serialized collections must be loaded and saved consistently, while generated transaction IDs must remain unique after reload.
- Validation: missing members, unavailable books, duplicate IDs, and invalid choices require controlled error handling.
- Interface integration: CLI and Swing needed to use the same Library business layer.
- Analytics: reports combine information across books, members, and transactions.
- GUI layout: dashboard cards, tables, navigation, and actions had to fit into a compact desktop window.

## LEARNING AND KEY TAKEAWAYS

- Applied object-oriented design to a complete real-world domain model.
- Practiced inheritance, abstraction, polymorphism, interfaces, and encapsulation.
- Gained practical experience with Java collections and stream-based filtering.
- Understood file streams, object serialization, and local persistence.
- Practiced custom exception handling and validation.
- Applied multithreading through Thread-based report generation.
- Learned to separate UI code from reusable business logic and integrate CLI and Swing interfaces.

## **FUTURE ENHANCEMENTS**

- Authentication: add librarian/admin accounts and role-based permissions.
- Reservations: implement wait-lists for unavailable books.
- Notifications: add reminders for due and overdue books.
- Barcode/QR: accelerate book identification and circulation.
- Database migration: use JDBC-backed storage for a multi-user deployment.
- Advanced analytics: add borrowing trends, category demand, and member activity charts.
- Concurrency: introduce stronger synchronization for simultaneous operations.
- Automated testing: add JUnit unit and integration tests.

## **CONCLUSION**

LibraX demonstrates how a practical library workflow can be implemented using Core Java without depending on an external database framework. The project combines object-oriented modeling, collections, interfaces, inheritance, exception handling, file I/O, serialization, multithreading, and Swing into one modular application. It supports book and member management, circulation, search, persistence, and analytics while providing visible evidence through both CLI and dashboard interfaces.

The current implementation establishes a foundation for future authentication, reservations, notifications, database-backed deployment, richer analytics, and automated testing. As an academic Core Java project, it connects syllabus concepts to a complete application rather than isolated programming examples.

## REFERENCES

- Oracle Java Documentation — Java SE APIs and language documentation.
- Oracle Java Tutorials — Collections, Exceptions, I/O, Serialization, Swing, and Concurrency.
- Java Platform documentation for `java.time`, `java.util`, `java.io`, and `javax.swing`.
- VITyarthi — Build Your Own Project: Project instructions and submission guidelines.
- LibraX project source code and implementation developed for the academic project.



THANK  
YOU

